

SynapseQuill — Recorrido Técnico de los Procesos

Cómo se encadenan los datos en cada flujo, paso a paso

Junio 2026

Cómo leer este documento

Este documento sigue **el dato real** a través de cada flujo del sistema: qué función lo recibe, qué hace con él, qué produce, y a qué función se lo entrega. No son definiciones sueltas, sino el **recorrido encadenado** (“esto se conecta con esto otro, que a su vez alimenta a aquello”).

Cada flujo trae:

- Un **diagrama de cadena** (cajas y flechas) que se lee de arriba abajo.
- El **desglose paso a paso**: para cada eslabón, *qué entra* → *qué hace* → *qué sale*, el archivo y función exactos, y **por qué** está hecho así.

Convención de las flechas: `entrada` → `[lo que hace]` → `salida`. Los nombres en `código` son funciones o archivos reales del proyecto.

Flujo 1 · Vídeo de un partido

Pregunta que responde: ¿cómo se pasa de “Valencia 2 - Barcelona 1” a un vídeo narrado en YouTube?

El orquestador es `run_match()` en `pipeline/runner.py`. El objeto que viaja de principio a fin es un `Match` (una ficha de datos del partido), del que se van derivando la narración, la voz, las imágenes y, al final, el `.mp4`.

La ficha `Match` (el dato central)

Todo gira en torno a esta estructura (`pipeline/match_monitor.py`):

```
Match
├─ home, away ..... nombres de los equipos
├─ home_goals, away_goals  marcador
├─ competition, date, venue, city, country
├─ goals: [ Goal ] ..... cada gol: player, team, minute, kind, description
├─ cards: [ Card ] ..... cada tarjeta: player, team, minute, color
```

Diagrama de cadena



Paso a paso

0 · Enriquecer datos — `enrich()` (`espn_enrich.py`)

- **Entra:** un `Match` que quizá solo tiene el marcador (2-1) pero no quién marcó.
- **Hace:** llama a la API pública de ESPN buscando ese partido por nombres de equipo y fecha (prueba hoy y los 3 días anteriores). De cada gol saca jugador, minuto, tipo (normal/penalti/en propia) y una **descripción** de la jugada.
- **Sale:** el mismo `Match`, ahora con `goals[]` lleno.
- **Por qué:** sin esto la narración sería vaga. Es *best-effort*: si ESPN falla, el pipeline continúa sin romperse (va envuelto en `try/except`).

Ejemplo. Entra `Valencia 2 - Barcelona 1`. Sale el mismo partido + `goals`: *López (min 12, normal), Pérez (min 70, penalty) — “remate de zurda desde el borde del área”*.

1 · Escribir la narración — `narrate()` (`narrator.py`)

- **Entra:** el `Match` enriquecido.
- **Hace:** construye un **bloque de hechos** (`_facts_block`) — una lista cronológica que fusiona goles y tarjetas por minuto — y se lo da a un **LLM** con instrucciones de actuar como relator: gancho dramático, 90-150 palabras, gritos de gol de exactamente 3 vocales, gramática española correcta, y la regla estricta de **usar solo los hechos dados**. Llama a `call_llm(..., max_tokens=600)`.
- **Sale:** `narration` (string, el guion hablado).
- **Por qué:** se le pasan **solo** los datos factuales + la personalidad de marca; que no invente lo garantiza el siguiente paso.

2 · Guardrail — `verify()` (`guardrail.py`)

Dos capas encadenadas:

```
narration → Capa 1: facts_check() → ¿el marcador del texto = marcador real?
                                     (regex determinista, coste cero)
      → Capa 2: llm_judge() → un 2º LLM responde JSON:
                                     {grounded, language_ok, tone_ok, reason}
      → passed = capa1 OK Y grounded Y language_ok
```

- **Si no pasa:** se **reescribe una vez** (se vuelve a llamar `narrate` añadiendo *“Be strictly factual”*) y se re-verifica. No hay tercer intento.
- **Por qué dos capas:** la regex pilla el peor fallo (marcador inventado) gratis; el juez IA cubre lo que la regex no puede (goleador inventado, idioma, tono).

Ejemplo. Si la narración dijera “gol de Messi” y Messi no está en los datos, el juez lo marca como `grounded: false` → se regenera.

3 · Pulir — `polish()` (`text_polish.py`)

- **Entra:** la narración verificada.
- **Hace:** primero arregla fallos típicos del español con reglas fijas (“la penalty” → “el penalti”, “de el” → “del”); luego un **LLM editor** reescribe frases que suenan raras **sin tocar los hechos**. Después **re-verifica** (solo la capa determinista): si el editor cambió un dato por error, se **descarta** su versión y se conserva la original.
- **Por qué:** un texto que dice “la penalty” delata a una máquina; pero pulir nunca debe alterar un marcador o un nombre, de ahí la re-verificación.

4 · Metadata de YouTube — `youtube_metadata()` (`narrator.py`)

- **Hace:** un LLM genera `title` (≤90 caracteres, con el marcador) y `description` en JSON. Las **etiquetas** se construyen **sin IA**, deterministas: competición, equipos, países, goleadores, estadio, ciudad.
- **Sale:** `meta = {title, description, tags}`.
- **Por qué:** las etiquetas (SEO) no se dejan al azar del LLM; se calculan exactas.

5 · Imágenes de ambiente — `build_visuals()` (`media_provider.py`)

- **Hace:** solo si “flux” está activado. Pide a **FLUX** (modelo texto→imagen, vía Together/Fal) una grada con los colores del equipo ganador (nunca escudos, para evitar copyright). En empate, genera las dos aficiones.
- **Sale:** `images[]` (rutas a PNG). Puede ir vacío y no pasa nada.
- **Por qué:** da un fondo con ambiente; es opcional porque por defecto el vídeo usa gráficos animados propios.

6 · Voz y subtítulos — `synthesize()` (`voice_generator.py`)

```
narration → limpia "G000L"→"GOL" (TTS se traba con vocales largas)
            → detecta gritos (i! y MAYÚS) → sube energía (+6% velocidad, +6Hz tono)
            → motor TTS (Edge-TTS / ElevenLabs)
            → audio.mp3 + tiempos de cada palabra (para subtítulos)
```

- **Por qué se limpian las vocales:** los motores (sobre todo ElevenLabs) tropiezan al sostener “Gooool”. El subtítulo conserva el texto original; solo el audio se limpia.
- **Por qué el “boost”:** Edge-TTS no tiene emociones, así que la euforia se simula subiendo velocidad y tono cuando el texto grita.

7 · Montar el vídeo — `assemble()` (`video_assembler.py`)

- **Entra:** `images` , `audio.mp3` , `subtitles` , `meta` , el `Match` .
- **Hace:** la **duración la marca la voz** (`total = audio.duration`). Dibuja con Pillow, frame a frame: marcador, escudos y una cronología “MINUTO A MINUTO” de goles y tarjetas que se revelan uno a uno. Encima, subtítulos estilo **karaoke** (una palabra grande amarilla cada vez, sincronizada con la voz). MoviePy une todo.

- **Sale:** `match_<id>.mp4` (vertical 1080x1920 o horizontal 1920x1080).

8 · Subir a YouTube — `upload_youtube()` (`publishers.py`)

- **Hace:** solo si lo pides o tienes auto-subida. Usa el permiso OAuth guardado. En modo práctica sube como **privado**. Si falla, el vídeo **no se pierde**: queda en la Biblioteca.

Flujo 2 · Laboratorio · Ciencia (RAG sobre arXiv)

Pregunta que responde: ¿cómo explica un tema científico **sin inventar**, apoyándose en papers reales? Aquí es donde entran los **embeddings**.

Diagrama de cadena



Paso a paso (con foco en los embeddings)

1 · ¿Construir o reutilizar? — `_topic_dir()` (`arxiv_rag.py`)

Cada tema tiene **su propia carpeta de índice**, nombrada con un slug legible + un hash corto (p. ej. `expected-goals-xg_a1b2c3d4`).

- **Por qué uno por tema:** si todos los temas compartieran un índice, preguntar por “detección de highlights” devolvería papers de “xG” (los que se indexaron primero). Separándolos, cada búsqueda solo ve papers de su tema. Se construye una vez y se cachea.

2 · Bajar papers — `_load_arxiv_docs()` (`arxiv_rag.py`)

- **Hace:** baja **hasta 8 artículos** de arXiv. Busca primero la frase exacta dentro del *abstract* (`abs:"..."`), y si no hay resultados, afloja la búsqueda.
- **Importante:** el “paper” aquí es **título + autores + abstract**, no el PDF entero (suficiente para divulgar, y mucho más rápido).
- **Por qué la frase exacta:** buscar texto libre devuelve papers que solo comparten una palabra común (“deep learning”) en temas no relacionados.

3 · Trocear — `RecursiveCharacterTextSplitter` (`arxiv_rag.py`)

- **Hace:** corta cada paper en **trozos de 1000 caracteres**, con **150 de solape** entre trozos contiguos. Intenta cortar por párrafos/frases, no a mitad de palabra.
- **Por qué trocear:** un vector por paper entero “promedia” demasiado y pierde precisión. Trozos pequeños dan vectores más exactos y permiten recuperar el fragmento concreto.
- **Por qué el solape:** si una idea cae justo en el corte, el solape de 150 evita que se parta y se pierda entre dos trozos.

4 · EMBEDDINGS — `bge-small` (`arxiv_rag.py`)

Este es el corazón del flujo. Conviene entenderlo bien:

- **Qué texto se convierte:** el contenido de cada trozo (título + autores + abstract, recortado a ≤ 1000 caracteres).
- **En qué se convierte:** una **lista de 384 números** (un “vector”) que representa el *significado* de ese trozo. El modelo es `BAAI/bge-small-en-v1.5`, de BAAI (instituto de Pekín), y corre **en tu ordenador**, gratis, sin internet.
- **Para qué sirven esos números:** permiten **buscar por significado**. Dos textos que hablan de lo mismo tienen vectores **cercanos** aunque no compartan palabras.

Ejemplo. El paper dice “*shot quality metric*” y tú preguntas por “*expected goals*”. No comparten ninguna palabra, pero sus vectores quedan cerca porque significan casi lo mismo. Una búsqueda por palabras fallaría; la de embeddings, no.

5 · Guardar en Chroma — `Chroma.from_documents()` (`arxiv_rag.py`)

- **Hace:** **Chroma** es la base de datos vectorial. Guarda, por cada trozo, su texto
 - su vector + de qué paper viene. Se persiste en disco para no reconstruir.

6 · Recuperar — `retrieve()` (`arxiv_rag.py`)

- **Hace:** tu **pregunta** se convierte en un vector **con el mismo** `bge-small` (imprescindible: pregunta y trozos deben vivir en el mismo “espacio numérico”). Chroma devuelve los **4 trozos más cercanos** a la pregunta.
- **Por qué k=4:** suficiente contexto sin meter ruido ni alargar demasiado el prompt.

7 · (Opcional) Graph RAG — `graph_rag.py`

- **Hace:** un LLM lee los trozos y extrae **relaciones** (“A causa B”) que guarda en un grafo (NetworkX, en memoria). Luego, partiendo de las palabras de tu pregunta, recorre los vecinos del grafo y añade esas relaciones como contexto extra.
- **Por qué:** el RAG normal recupera *texto*; el grafo añade *relaciones explícitas* entre conceptos. Es *best-effort*: si falla, simplemente no se añade.

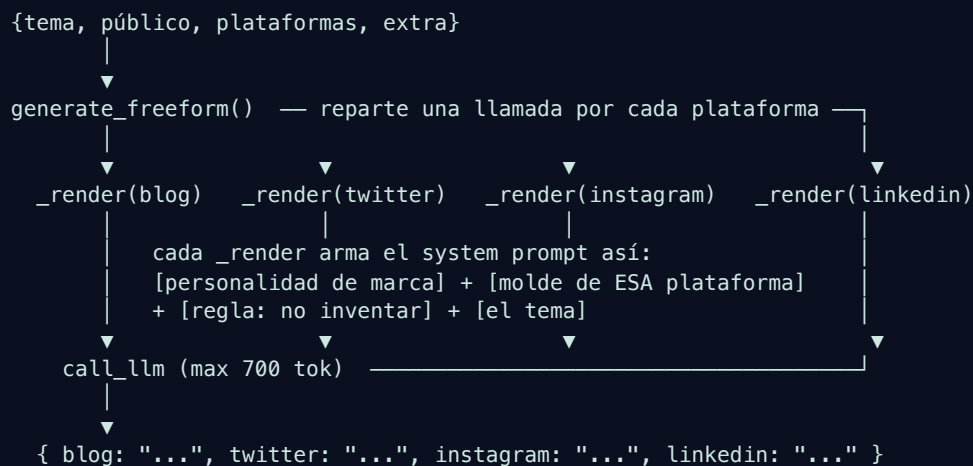
8 · Redactar — `call_llm()` (`arxiv_rag.py`)

- **Hace:** se le pasan al LLM **solo** los 4 trozos (+ relaciones) y se le ordena: “*explica usando ONLY el contexto dado; no inventes hechos ni citas*” (`max_tokens=900`).
- **Por qué RAG y no responder “de memoria”:** así la respuesta se basa en papers reales recién descargados (actualizados y citables), no en lo que el modelo “recuerde”, que puede estar desfasado o ser inventado.

Flujo 3 · Crear (texto multiplataforma)

Pregunta que responde: ¿cómo se genera, de un tema, un texto distinto y a medida para blog, X, Instagram y LinkedIn?

Diagrama de cadena



Paso a paso

1 · Repartir por plataforma — `generate_freeform()` (`content_generator.py`)

- **Entra:** `{topic, audience, platforms, extra}` + (del perfil) idioma, personalidad de marca y proveedor LLM.
- **Hace:** para cada plataforma elegida, llama al núcleo `_render`.
- **Sale:** un diccionario `{plataforma: texto}`.

2 · El núcleo — `_render()` (`content_generator.py`)

Arma el *system prompt* en este orden exacto:

1. **La personalidad de tu marca** (`system_preamble` del perfil) — primero de todo.
2. `"Escribes contenido de {plataforma} en {idioma}."`
3. **El molde de esa plataforma** (ver tabla).
4. **La regla de no inventar.**
5. `"Devuelve solo el texto del post."`

Luego llama `call_llm(..., max_tokens=700)`.

Los moldes exactos por plataforma:

Plataforma	Formato	Longitud	Hashtags
Blog	SEO, título H1 + 2-3 párrafos	250-400 palabras	—
Twitter/X	un solo tweet, directo	máx. 280 caracteres	2-3
Instagram	caption vívido y emocional	~150 palabras	5-8 (última línea)
LinkedIn	profesional, tono respetuoso	~120 palabras	3-4

- **Por qué un solo núcleo:** el mismo `_render` sirve también para el modo “partido” (texto atado a los hechos del encuentro); lo único que cambia es el molde y la regla de fundamentación.

Ejemplo. Tema “la táctica del fuera de juego”, plataformas *X + LinkedIn* → un tweet corto con 2-3 hashtags y, aparte, un post de LinkedIn de ~120 palabras más reposado. Mismo tema, distinto molde.

Flujo 4 · Laboratorio · Finanzas

Pregunta que responde: ¿cómo da el precio y los titulares de una empresa, y qué pasa si una fuente falla?

Diagrama de cadena

```
ticker ("NVDA")
|
v
market_summary()
|
+-- quote() -----> [ ¿hay clave Finnhub?
|   |             |
|   |             +-- sí → Finnhub /quote — precio "c" = 0? → (miss)
|   |             |   precio válido → usa c (precio) y dp (% día)
|   |             +-- miss / sin clave → yfinance: last_price,
|   |             |   previous_close → % = (precio-prev)/prev*100
|   |
|   +-- company_news() → Finnhub /company-news → 5 titulares
|       (sin clave → lista vacía, NO rompe)
|
v
"NVDA: $203.26 (-1.53% hoy)
- titular 1
- titular 2 ..."
```

Paso a paso

1 · Precio — `quote()` (`finance.py`)

- **Hace:** pregunta primero a **Finnhub**. Si Finnhub devuelve precio 0 (señal de ticker desconocido) o no hay clave, cae a **yfinance** (Yahoo, sin clave) y calcula la variación del día comparando con el cierre anterior.
- **Por qué el fallback:** así el precio casi nunca falla, aunque no tengas clave de Finnhub.

2 · Titulares — `company_news()` (`finance.py`)

- **Hace:** pide a Finnhub las **5 noticias** más recientes (última semana). **Sin clave, devuelve lista vacía** en lugar de romper.
- **Por qué:** el resumen de precio sigue funcionando aunque no haya titulares.

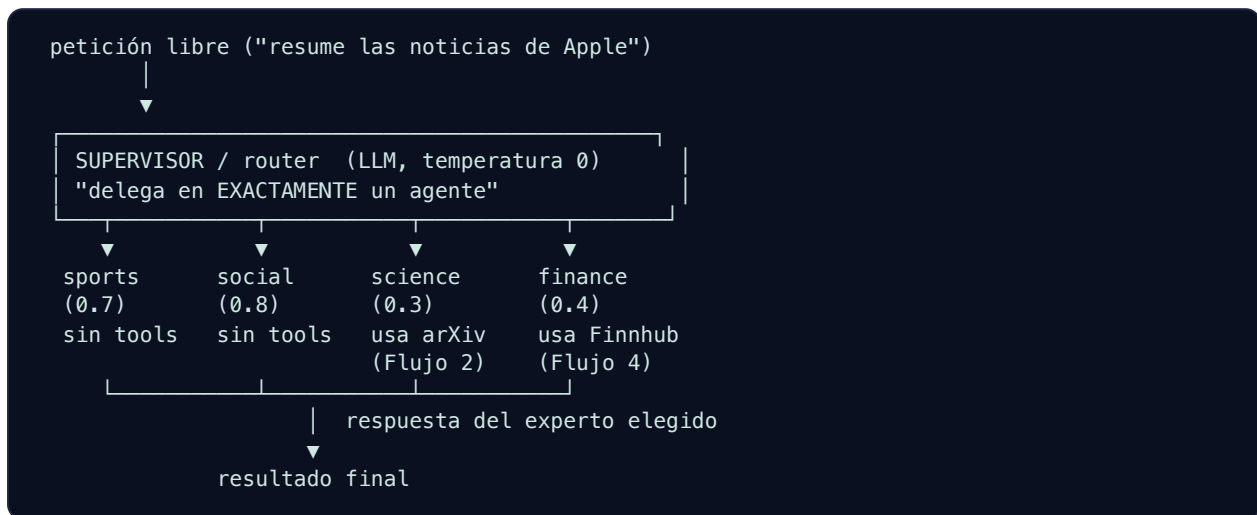
3 · Ensamblar — `market_summary()` (`finance.py`)

- **Hace:** junta la línea de precio + las viñetas de titulares en un texto.
- **Nota:** esta misma función es la **herramienta** que usa el agente de finanzas (Flujo 5).

Flujo 5 · Laboratorio · Agentes (LangGraph)

Pregunta que responde: ¿cómo decide un “jefe” qué experto atiende tu petición?

Diagrama de cadena



Paso a paso

1 · El router — `build_supervisor()` (`agents/graph.py`)

- **Hace:** un LLM con **temperatura 0** (decisión estable, sin creatividad) lee tu petición y elige **un solo** experto según un prompt que describe a cada uno.

2 · Los 4 expertos

Agente	Temperatura	Herramienta	Para qué
sports	0.7	ninguna	contenido factual de fútbol
social	0.8	ninguna	copys para redes (más creativo)
science	0.3	arXiv RAG (Flujo 2)	explica ciencia, siempre se fundamenta
finance	0.4	Finnhub (Flujo 4)	cifras de mercado reales

- **Por qué distintas temperaturas:** social necesita creatividad (0.8); ciencia y finanzas deben ser factuales (0.3-0.4); el router debe ser determinista (0).
- **Conexión clave:** los agentes *science* y *finance* **reutilizan** los Flujos 2 y 4 como herramientas. No duplican lógica: el agente de finanzas llama exactamente al mismo `market_summary()` del Flujo 4.

3 · El modelo y su red de seguridad — `get_llm()` (`core/llm/`)

- Todos los agentes corren sobre el proveedor configurado, con **cadena de respaldo automática**: si el principal falla, prueba el siguiente (groq → gemini → cerebras → ollama, usando solo los que tengan clave).

Ejemplo. Escribes “*escribe un tweet de la final*” → el router lo manda a *social*. Escribes “*resume las noticias de Apple*” → va a *finance*, que llama a Finnhub y devuelve el resumen.

Resumen: cómo se conectan los flujos

- **Crear (3)** y **Agentes (5)** comparten la capa de modelos de texto, pero Crear usa llamadas directas y Agentes usa LangChain (con red de respaldo).
 - **Finanzas (4)** es una **subrutina** de Agentes (5): el agente *finance* la usa como herramienta.
 - **Ciencia (2)** alimenta a Agentes (5): el agente *science* la usa como herramienta.
 - Las tres peticiones del Laboratorio (ciencia, finanzas, agentes) y la de Crear se **guardan en el historial** del perfil.
-

Documento técnico generado para SynapseQuill · FIFA World Cup 2026. Todas las cifras (8 papers, trozos de 1000/150, k=4, temperaturas, tokens) están tomadas del código real del proyecto.